

WHAT THIS SESSION IS

The first coding class, run on 23 August 2026. One project is created with uv, one API key goes into a `.env` file, and then the same question is sent to the same model three different ways: by hand with the REST API, with the official OpenAI SDK, and finally with LangChain. Doing it in that order is the whole point of the session, because it shows exactly what each layer takes off your hands. The class then adds a system prompt to control the model's behaviour, and builds conversation memory by hand so you can watch the token count climb. Fifteen examples were planned and four were built, which is a fair trade for a first technical session.

CONTENTS

1. Where this session picks up
2. What you are actually building
3. Why Python, and why it is not the only option
4. Setting the project up with uv
5. The API key, and where it goes
6. Application 1: the hard way, raw REST
7. Application 2: the same call with the OpenAI SDK
8. The date demo, and what a harness is
9. Application 3: the same call with LangChain
10. Application 4: system prompt, and controlling the tone
11. Application 5: memory by hand, and what it costs
12. The three approaches side by side
13. Clarifications from the Q&A
14. Key takeaways

1 Where this session picks up

The previous class was a summary of the recorded foundations and a preview of what the course leads to. Nothing was built. This one starts the actual journey, and it starts from scratch: an empty folder, a terminal, and a fresh project.

The pace was set out again at the start, and it is worth repeating because it shapes how you should use these notes.

HOW TO WORK THROUGH A PRACTICAL CLASS

Do not type along during the live session unless you have two screens and you are fast. Fifty or a hundred people all coding at different speeds turns three hours into two examples. Watch the demonstration, note your doubts, and repeat everything afterwards with the recording, the code files and these notes, all of which land on the dashboard within four to eight hours. Recordings play fine at 2x if you are revisiting something you already saw.

2 What you are actually building

Start from an ordinary application. It could be written in Java, Python, C#, JavaScript, anything. Input goes in, business logic runs, output comes back, and every branch of it was coded by a developer. Predictable, and completely deaf to plain English.

What users now expect is different. Somebody should be able to type "I joined a gym recently, suggest me some good shoes" and get a sensible answer, without picking from dropdowns or guessing the right keyword. For that, the application has to get smarter.

You do not make it smarter by training a model. Plenty of trained models already exist, owned by OpenAI, Anthropic, Google and others, and those companies have also built applications around them: ChatGPT, Claude, Gemini, Copilot, DeepSeek, Grok. Your job is to bring the same kind of capability into your own application.

ONE DISTINCTION TO KEEP STRAIGHT

Your Python code does **not** connect to ChatGPT. ChatGPT is an application, the same category of thing you are building. Your code connects directly to the trained model behind it, over an API, the same way you would integrate a payment gateway or a weather service. Somebody else owns that service, so you pay for it and you carry a key.

3 Why Python, and why it is not the only option

Every mainstream language can do this. Telusko runs the same course in Java and Spring Boot in the weekend morning batch, and that batch is on its third run.

Python leads for one reason: volume. It has the most libraries, the largest community of people who have already experimented with this, and therefore the most features and the most support when something breaks. The comparison drawn in class was with Java on distributed enterprise systems, where the community

weight sits the other way round. Pick the ecosystem where the problems you will hit have already been hit by somebody else.

4 Setting the project up with uv

Two things are needed before starting: an IDE, and Python installed. The class used VS Code. Any editor is fine, and being fussy about IDEs is not a useful habit for a developer.

Why a package manager at all

When you install something globally on your machine, it is available to every project on that machine. That sounds convenient until two projects want different versions of the same library. A virtual environment gives one project its own isolated set of tools and versions, so upgrading something for project two cannot quietly break project one.

uv is what this course uses to manage that. The comparison offered in class was Maven or Gradle in the Java world, with one important correction that came up again in the Q&A: Maven does not create a virtual environment, uv does. Python also does not strictly require a project manager the way Java does, but once you want an isolated environment plus managed dependencies in one tool, uv earns its place.

UV VERSUS PIP, IN ONE LINE

pip is a package installer. uv is a package installer plus a project and dependency manager that also creates and uses the virtual environment for you. It is written in Rust, which is why it is noticeably faster. Official documentation: <https://docs.astral.sh/uv/>

Installing uv

First, check what is already on the machine. This part is the same everywhere.

```
python --version
uv --version      # if this errors, uv is not installed yet
```

Windows, the way it was done in class

```
python -m pip install --user uv
```

Installing it this way puts the executable inside your user Scripts folder, so the shell will not find **uv** until that folder is on your PATH. In class this was fixed by opening System Properties, then Environment Variables, then editing Path under System variables and adding the full path to that Scripts folder. It usually looks like this, with your own Windows user name in place of the one shown:

```
C:\Users\<your-user-name>\AppData\Roaming\Python\Python3xx\Scripts
```

The part from `AppData` onwards is the same for everyone. The beginning depends on your machine, so copy it from your own system rather than from the screen. Close the terminal and open a new one afterwards, otherwise it will still be using the old PATH.

macOS, and Linux

The class was demonstrated on Windows, so here are the equivalents. Open Terminal and use whichever of these suits you.

```
# option 1: the official installer, recommended by Astral
curl -LsSf https://astral.sh/uv/install.sh | sh

# option 2: with Homebrew, if you already use it on a Mac
brew install uv

# option 3: the same route as the class, through pip
python3 -m pip install --user uv
```

The installer script puts `uv` in `~/.local/bin`. If `uv --version` still says command not found afterwards, that folder is not on your PATH yet. Add it to your shell profile once:

```
# zsh, which is the default on macOS since Catalina
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc

# bash, common on Linux
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Homebrew users do not need this step, since `brew` installs into a location that is already on the PATH. If you are on an Apple Silicon Mac and installed `uv` through pip, note that the pip user folder is `~/Library/Python/3.x/bin` rather than `~/.local/bin`, so put that on the PATH instead.

For completeness, Windows has an official installer too, which does the PATH work for you:

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

THE TEMPORARY VERSION OF THE SAME COMMAND

If you install uv from inside the VS Code terminal without setting PATH, it works only in that terminal. Close the terminal or reopen the project and `uv` is gone again. Setting the environment variable once is the version worth doing. If `uv --version` still errors after that, open a fresh terminal first, and then debug it the way you would debug anything else. Being able to sort out a PATH problem is a reasonable expectation of a developer, on any operating system.

Creating the project

Create an empty folder, open it in VS Code, open a terminal at that folder, and then:

```
# 1. create the project scaffolding
uv init

# 2. create the virtual environment folder (.venv)
uv venv

# 3. activate it. uv prints the exact line for your shell in green,
#    copy that one. it differs by platform:

.venv\Scripts\activate          # Windows PowerShell
.venv\Scripts\activate.bat      # Windows, old command prompt
source .venv/bin/activate       # macOS and Linux
```

Activation is a convenience rather than a requirement. `uv run yourfile.py` always uses the project's environment whether or not you activated it, and `uv add` will create `.venv` on its own if you skipped step two. Activate if you like seeing the environment name in your prompt and being able to type `python yourfile.py` directly. To leave it again, on any platform, type `deactivate`.

`uv init` produces `pyproject.toml`, a `.python-version`, a `.gitignore` and a starter file. `pyproject.toml` is where the project's dependencies are declared, which is the role `pom.xml` plays in a Maven project. `uv venv` then creates the `.venv` folder itself, full of library files and scripts. You never need to open it.

Adding libraries

The equivalent of `pip install` is `uv add`, and it does two things: installs the package into this project's environment, and records it in `pyproject.toml`.

```
# what the first example needs
uv add requests python-dotenv

# added later in the class, when each was needed
uv add openai
uv add langchain langchain-openai
```

One environment was used for the entire class. Creating a fresh virtual environment for every small demo file is not how anybody actually works, so all five example files live side by side in the same project.

Running a file

```
# works everywhere, activated or not
uv run telusko-app1.py

# or, if you activated the environment first
python telusko-app1.py      # Windows
python3 telusko-app1.py     # macOS and Linux, if plain python is Python 2
```

IF YOU ARE PICKING THIS PROJECT UP ON ANOTHER MACHINE

Everything the project needs is already declared in `pyproject.toml` and pinned in `uv.lock`. So on a fresh clone you do not repeat the install steps one at a time, you run `uv sync` once and uv recreates the same environment with the same versions. Then add your own `.env`, because that file is deliberately never committed.

For reference, this is what `pyproject.toml` looked like by the end of the session:

```
[project]
name = "pythonlive"
version = "0.1.0"
description = "Talking to LLM - class demo code"
readme = "README.md"
requires-python = "≥3.14"
dependencies = [
    "langchain≥1.3.16",
    "langchain-openai≥1.6.0",
    "openai≥3.3.1",
    "python-dotenv≥1.2.3",
    "requests≥2.34.2",
]
```

5 The API key, and where it goes

Why OpenAI for this batch

The class settles on OpenAI, and the reasoning was given rather than assumed. OpenAI supplies chat models, **embedding models**, image models and audio models under one key. The embedding models matter, because RAG later in the course needs them. Anthropic, for example, does not offer its own embedding models, so a RAG build on Anthropic means bringing an embedding model from somewhere else. That is entirely doable, it is just one more moving part in a first course.

This is not exclusive. Gemini, Anthropic and local models will all be demonstrated in later classes, and switching providers is a small change once you are working through a framework.

Getting the key

1. Go to platform.openai.com and open the API platform, which is separate from the ChatGPT product.
2. Add credit. Around **\$5** comfortably covers this entire course. If you push hard, \$10 to \$15. The demos in class use a small model precisely so the tokens cost almost nothing.
3. Open **API keys** in the left menu, then create a new secret key with any name you like.
4. **Copy the key immediately.** Once you click Done the key cannot be shown again. If you lose it, revoke it and make a new one.

TURN AUTO RELOAD OFF, AND TREAT THE KEY LIKE A PASSWORD

The billing page has an auto reload setting that tops your balance up automatically. Leave it off. A leaked key with auto reload on is an open tab on your card. If a key does leak, revoke it from the same page and it is dead immediately, so nobody can use it even if they have the string. The trainer revoked his own key on air at the end of the class, which is the right instinct after showing one on screen.

The .env file

Create a file called exactly `.env`, with no name in front of the dot, at the root of the project, next to your Python files and not inside `.venv`. Getting this in the wrong folder was the first small mistake of the class and a student caught it in chat.

```
# .env (at the project root, never committed)
OPENAI_API_KEY=sk-your-key-here
```

ON HARD CODING, ASKED LIVE AND ANSWERED STRAIGHT

A student pointed out that keys should not be hard coded and pushed to Git. Correct, and the answer given was worth keeping: this class is not teaching development practice, it is teaching how to reach the model. Production key management is a real topic and it comes later in the course. For now, keep the key in `.env`, keep `.env` in `.gitignore`, and commit a `.env.example` with a placeholder so that anyone cloning the project knows what to fill in.

6 Application 1: the hard way, raw REST

The first file talks to the model with nothing but `requests`. No SDK, no framework. Every piece of the HTTP call is written out by hand, and that is the point: you will not appreciate what the next two layers do until you have assembled this once.

The four things a request carries

YOUR PYTHON PROGRAM

```
|
| 1. WHERE      the URL
|      https://api.openai.com/v1/chat/completions
|
| 2. WHO        the headers
|      Content-Type: application/json
|      Authorization: Bearer sk-....
|
| 3. WHAT       the payload (JSON body)
|      { "model": "...", "messages": [ ... ] }
|
| 4. HOW        requests.post(...)
v
```

+-----+

```
|  OpenAI, the provider  |
|  many trained models  |
|  inside one address   |
+-----+
```

|

v

the response JSON

the address gets you to the building.

the header key opens the door.

the payload says which model to talk to and what to ask.

That analogy was used in class and it is a good one. A key on its own is useless unless you also know which building it opens, and getting inside is useless unless you say who you want to speak to and what you want to ask.

telusko-app1.py

```
import os
import requests
from dotenv import load_dotenv

load_dotenv()

api_key = os.getenv("OPENAI_API_KEY")

url = "https://api.openai.com/v1/chat/completions"

headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {api_key}"
}

payload = {
    "model": "gpt-4o-mini",
    "messages": [
        {
            "role": "user",
            "content": "What is AI in short"
        }
    ]
}

response = requests.post(
    url,
    headers=headers,
    json=payload
)

print("-----")
print(response.json()["choices"][0]["message"]["content"])
print("-----")
print(response.json())
```

Line by line

- `load_dotenv()` reads the `.env` file and puts its values into the process environment. Without this line, `os.getenv` finds nothing.

- `os.getenv("OPENAI_API_KEY")` pulls the key out into a variable. The key never appears in the source file.
- `url` is the destination. Note the path carefully, more on this below.
- `headers` carries two things. `Content-Type` tells the server the body is JSON. `Authorization: Bearer <key>` is how the key travels. A request can carry information in the URL, in the headers or in the body. The URL is the least private of the three, which is why the key goes in a header.
- `payload` is the body. `model` picks which trained model answers. `messages` is a list, and each message has a **role** and **content**. Here the role is `user`, meaning you. There is also a `system` role for setting behaviour, which shows up in section 10.
- `requests.post` sends it. POST rather than GET because the information travels in the body and the headers rather than in the URL.
- The two prints show the two useful views of the answer: just the text, and the whole JSON document.

HONEST STATUS FROM THE LIVE CLASS: A 400 ON THE FIRST RUN

The first run failed with `400 Bad Request` and a message about a missing required parameter, `prompt`. The cause was the URL. It had been typed as `/v1/completions`, which is the older completions endpoint and expects a `prompt` field, not a `messages` list. Changing it to `/v1/chat/completions` fixed it immediately.

Worth reading the status code properly: a 400 means you reached the server and the server rejected what you sent. Compare with a 401, which means your key was refused, and a 404, which means the address does not exist at all. Getting a 400 is actually good news, because it proves the network path and the key are fine and the problem is in your request.

What comes back

The response is not just the answer. Printing the full JSON shows the reply text, which model actually served the request, and a usage block with the input tokens, the output tokens and the total. That metadata is available on every call, from every approach shown in this class, and it is how you keep an eye on cost.

```
# the reply text only
response.json()["choices"][0]["message"]["content"]

# everything, including which model answered and what it cost
response.json()
```

WHERE THE PARAMETER NAMES COME FROM

A fair question was asked in class: how would anyone know the body needs `model` and `messages` ? From the provider's own documentation. Every API you integrate publishes what it expects, and reading it is part of the job.

OpenAI API reference: platform.openai.com/docs/api-reference

7 Application 2: the same call with the OpenAI SDK

Everything in application 1 is plumbing, and it is the same plumbing on every call. If that were the only way to reach a model, most developers would have walked away. So the providers ship an SDK that writes the plumbing for you.

```
uv add openai
```

telusko-app2.py

```
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()

client = OpenAI()

response = client.responses.create(
    model="gpt-4o-mini",
    input="tell me todays current date",
)

print(response.output_text)
```

Compare it with the first file. No URL. No headers. No JSON body. No manual POST. The `OpenAI()` client picks the key up from the environment on its own, builds the request, sends it, and hands back an object with the text already extracted.

THE ONE LINE THE DOCUMENTATION DOES NOT SHOW YOU

Copy the snippet straight from the official docs and it fails with a complaint about a missing API key. The docs assume the key is already an environment variable on your machine. Because this project keeps it in a `.env` file, you have to add `load_dotenv()` yourself. That happened live, and it is a very common first stumble.

Asked about Telusko, this version answered correctly and described the YouTube channel and learning platform. The reply is the same quality as application 1, because it is the same model. Only the amount of code you wrote changed.

8 The date demo, and what a harness is

Before introducing LangChain, the class made the case for why a framework is needed at all, and it did it with one question.

The SDK file was changed to ask *tell me today's current date*. The answer came back as October 2023.

Now put that next to Gemini in a browser, which was asked the same thing in the previous class and answered with the correct date. Both are ultimately talking to a trained model with a cutoff. So how does one of them know?

HARNESS, DEFINED

The **harness** is everything you build around the model so that the model becomes useful. Tools, so it can fetch live information and take actions. Context and RAG, so it can answer from your data. Memory, so it can recall the conversation. Control, so it is allowed to do certain things and not others. The model is the brain. The harness is everything else, and it is what separates a demo from an application.



All the tools an application owns are registered with the model. The model is effectively told: do not answer only from what you know, here is what this application can do, look at the question and decide which of these to invoke. Ask about the World Cup and the sports tool fires. Ask for the date and the date tool fires. How the model makes that decision is a dedicated session later, on tool calling.

Once you accept that an application needs the model plus tools plus RAG plus memory plus possibly MCP, all working together, hand written HTTP calls stop being reasonable. Even the SDK stops being enough, because it is built around talking to one provider, not around orchestrating all of that. That is the gap a framework fills, and the most widely used one in Python is **LangChain**.

IS LANGCHAIN COMPULSORY?

No, and this was said clearly. You just built two working applications without it. LangChain does not add any capability that is otherwise impossible, and it does not magically give the model today's date either, that still needs tool calling. What it does is make the surrounding work far easier, which matters the moment your application is doing more than one thing. It is also not an alternative to FastAPI. FastAPI builds your web layer. LangChain builds your AI layer. Official documentation:

<https://docs.langchain.com/>

9 Application 3: the same call with LangChain

```
uv add langchain langchain-openai
```

Two packages. `langchain` is the core. `langchain-openai` is the provider specific piece that knows how to talk to OpenAI, which is why the class installs both. Swap the provider later and you swap this second package.

telusko-lc.py

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-4o-mini")

response = model.invoke("What is AI in short")

print(response.content)
```

That is the entire application. No URL, no headers, no body, no client construction. You name the model, you call `invoke` with your question, and you read `.content` off the reply.

The plumbing has not disappeared. The URL, the headers and the JSON body are all still built and sent, they are just built by LangChain instead of by you. That is what abstraction means here, and it is exactly the same trade as using a framework anywhere else.

WHY GPT-4O-MINI AND NOT THE NEWEST MODEL

Every demo in this class uses a small, cheap model on purpose. "What is AI in short" does not need a flagship model, and burning tokens on a warm-up example is waste. Which model you actually pick in a real project is a business decision: how good does the answer need to be, and how much is the company willing to spend per token. Both live in the same conversation.

10 Application 4: system prompt, and controlling the tone

So far every message has been the user's. There is a second kind, and it belongs to you, the person who owns the application.

The analogy given in class works well. Suppose you tell a colleague: my friend is going to come and ask you some things, and when he does, behave as my employee, not as my business partner. You have set the tone before the conversation starts. Your friend never sees that instruction. They just experience the behaviour.

- **SystemMessage** is that instruction. It sets role, tone, scope and rules, and the end user never types it.
- **HumanMessage** is what the user actually asks.
- **AIMessage** is what comes back.

```

from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage

load_dotenv()
model = ChatOpenAI(model="gpt-4o-mini")

messages = [
    SystemMessage(
        content="You are a python trainer. Answer in three short bullet points "
        "also answer only python related question if some one ask about "
        "java or ai or any other politely say i can answer only python queries"
    ),
    HumanMessage(
        content="tell me about python ?"
    )
]

response = model.invoke(messages)
print("Content :", response.content)
print("Tokens :", response.usage_metadata)
print("Model :", response.response_metadata.get("model_name"))

```

Notice that `invoke` now takes a list of messages instead of a plain string. Passing a string, as in the previous file, is simply a shortcut for a list containing one `HumanMessage`.

Does it actually hold the line?

Asked why to learn Python, it answered in bullet points as instructed. Then the human message was changed to ask about Java and Spring Boot, and the reply was:

```

I'm sorry, I can answer only Python related queries.
If you have any questions about Python, feel free to ask.

```

That is the system prompt doing its job. The behaviour of the model was controlled from your code, not by the person typing.

THIS IS A DEMO, NOT A SECURITY BOUNDARY

Stated plainly in class: there are better ways to do this and lots more around it. A system prompt shapes behaviour, it does not enforce it. Somebody determined enough can often talk a model past its instructions, which is the whole field of prompt injection, and that has its own module later in the course. Treat the system prompt as the first layer, not the only one.

Reading the metadata

The last two prints are worth keeping in every experiment you run.

```
print("Tokens   :", response.usage_metadata)
print("Model    :", response.response_metadata.get("model_name"))
```

On the run shown in class, the input was around 59 tokens, the output 21, and the total 80. The model name came back as `gpt-4o-mini`, confirming which model actually served the request.

11 Application 5: memory by hand, and what it costs

The model has no memory. Every call is independent. If you want it to remember the previous turn, you send the previous turn again.

Since `messages` is just a Python list, and the reply is itself a message object, you can append the reply back into the list and then add the next question.

```
messages.append(response)

messages.append(
    HumanMessage(
        content="Now give me the first thing I should build."
    )
)

response = model.invoke(messages)
print(response.content)
print("-----")
print("Tokens   :", response.usage_metadata)
```

The second question never mentions Python. It does not mention any topic at all. And the answer came back telling the learner to start with a hello world program and get familiar with Python syntax. The context carried over, because the whole conversation was sent again.

CALL 1

```
[ SystemMessage ]  
[ HumanMessage 1 ]
```

```
input tokens : 56  
total tokens : 118
```

same short question, more than double the bill,
because the history rides along every single time.

CALL 2

```
[ SystemMessage ] sent again  
[ HumanMessage 1 ] sent again  
[ AIMessage 1 ] sent again  
[ HumanMessage 2 ] new
```

```
input tokens : 136  
total tokens : 201
```

Those numbers came off the screen in class, and they are the clearest argument you will get for why memory management is a real engineering topic rather than a convenience. Left alone, a long conversation grows its own cost without anybody adding a feature.

THIS IS THE BEGINNER VERSION ON PURPOSE

Appending to a list is memory management with the lid off, so you can see exactly what is being sent. Production memory is different: trimming old turns, summarising them, storing conversations in a database, scoping them per user rather than per run. All of that is planned for later classes. A question was asked about whether memory is per session, and the answer was no, it is whatever you build. Nothing here is automatic.

12 The three approaches side by side

	Raw REST	OpenAI SDK	LangChain
You write	URL, headers, JSON body, the POST call, and the dig into the response	A client, the model name, the input	The model name and <code>invoke</code>
Install	<code>requests</code>	<code>openai</code>	<code>langchain</code> , <code>langchain-openai</code>
Key handling	You read it and set the header	Picked up from the environment	Picked up from the environment
Answer text	<code>["choices"][0]</code> <code>["message"]</code> <code>["content"]</code>	<code>response.output_text</code>	<code>response.content</code>
Token metadata	Yes, in the JSON	Yes	Yes, <code>usage_metadata</code>
Switching provider	Rewrite everything	Different SDK, different code	Change one import and one class
Tools, RAG, memory, MCP	All by hand	Partly, provider specific	What it is built for

The important row is the last one. If the job were only "send a question, get an answer", the SDK would be plenty and this course would be a weekend. The reason the industry converged on frameworks is everything underneath that row.

A NOTE ON THE PHRASE "AGENTIC"

An observation made in passing that is worth holding on to. Look at what happened across these three files. The work did not disappear between application 1 and application 3. It moved. Something else is building the request on your behalf. That shift, from you doing the steps to something else doing the steps for you, is the same shift agents represent, just at a much smaller scale.

13 Clarifications from the Q&A

- **Is LangChain like Django?** A student offered the comparison and it was accepted: Django helps you build web applications, LangChain helps you build AI applications on top of LLMs.
- **Is LangChain an alternative to FastAPI?** No. FastAPI is a web framework. LangChain is an agentic AI framework. They sit at different layers and you will use both.
- **Can we see token usage without LangChain?** Yes. All three approaches return the same metadata, because it comes from the provider, not the library. Application 1 printed it straight out of the raw JSON.

- **What is the difference between uv and pip, again?** pip installs packages. uv installs packages, creates the virtual environment, and manages the project's dependencies as a whole. It is also considerably faster.
- **Is uv the same as Maven?** Similar in spirit, not identical. Maven does not create a virtual environment. Also, Python does not require a project manager the way Java does, uv is there because isolation and dependency management are genuinely useful.
- **What made the model remember the earlier answer?** Nothing automatic. It was appended into the message list by hand. That is why the input token count jumped.
- **Do we need to run `uv venv` every time?** No. It creates the environment once for the project.
- **Can I use my company's data with OpenAI?** Not without approval. Anything sensitive, banking data in particular, generally cannot go to a closed hosted model. That is a large part of why local and open weights models matter, and it is covered later along with the security discussion. For learning, a personal key is fine.
- **Can I use a Gemini key instead?** Yes. It is already demonstrated in one of the recorded videos, and it will be shown in a live class too. Local models through Ollama were promised for an upcoming session.
- **How much will the key cost me?** \$5 of credit buys on the order of a million tokens on a small model, which is far more than this course needs.
- **Is React or Next.js taught?** No. A final application may use a front end, but front end development is not part of the syllabus.
- **Can we have a separate Q&A session?** Suggested by a student, and the team said they would consider it as a session outside the regular class.
- **Where do I get help between classes?** The private Discord. If you have not been added, contact the WhatsApp number on the course dashboard. Every video also has its own discussion section.

14 Key takeaways

- **Your code talks to the model, not to ChatGPT.** ChatGPT is an application in the same category as the one you are building.
- **One project, one virtual environment, one `.env`.** `uv init`, `uv venv`, activate, then `uv add` for every library. The only line that changes between Windows and macOS is the activate command: `.venv\Scripts\activate` against `source .venv/bin/activate`.
- **A request carries the destination in the URL, the key in a header, and the question in the body.** Write that out once by hand and every later abstraction makes sense.
- **Use `/v1/chat/completions`, not `/v1/completions`,** with a `messages` list. The older endpoint expects a `prompt` and will reject you with a 400.
- **The response is more than the answer.** Token usage and the model name come back on every call, from every approach.
- **The SDK removes the plumbing. LangChain removes more than that,** and is built for the tools, RAG, memory and MCP work that comes next.

- **The harness is everything around the model** that gives it live data, your data, memory and permission to act. That is what the course is actually about.
- **A system prompt sets behaviour before the user ever types.** It shapes the answer, but it is not a hard boundary.
- **Memory is you re-sending the conversation**, and the token count proves it. 56 input tokens became 136 for a second short question.
- **Small models for demos.** Model choice in a real project is a quality against cost decision.
- **Copy the key once, keep auto reload off, revoke anything that leaks.**

BEFORE THE NEXT CLASS

Practise what was built here: create the project yourself, get your own key in, and run all four files. Finish the remaining foundation recordings, particularly the prompt engineering section, since it is directly relevant from here on. If you joined late, watch the previous live session at 2x and skip to the parts you need. The code files, the board work and the notes for this session are on the dashboard under the live classes section, and a GitHub link follows.

COMING NEXT

More examples, and more of them per class now that the setup is out of the way. Temperature and what it controls, which was deliberately skipped here to avoid overloading a first technical session. Local models through Ollama, so that a key is not the only way in. Then further into LangChain, towards prompt templates, output parsers, chains, and eventually tool calling, which is where the date question finally gets a real answer.